

ロボット犬で学ぶ

Java & ロボティクス

コンピュータ授業のための教科書

OOP × ロボット × ROS を一貫したストーリーで学ぶ
全11課 + 付録

目次

第0課 Java によるこそ

第1課 クラスとオブジェクト

第2課 カプセル化

第3課 コンストラクタと初期化

第4課 継承

第5課 インターフェースによる抽象化

第6課 多態性（ポリモーフィズム）

第7課 戦略パターン—頭脳の差し替え

第8課 ロボットの頭脳を入れ替える

第9課 世界モデル—状態と行動の予測

第10課 publish / subscribe 通信

第11課 マルチエージェントと協調動作

付録A 用語集

付録B 演習問題解答

付録C コード索引

付録D 教師用ガイド

シラバス

シラバス

科目名

ロボット犬で学ぶ Java & ロボティクス (Java Programming through Robotics)

単位数

2 単位 (講義 15 回 + 演習)

対象

情報科学科 / 総合科学科 1～2 年次 Java 未経験可。プログラミング基礎を履修していることが望ましい。

授業計画

回	課	タイトル	内容
1	0	Java によるこそ	環境構築、Hello World、変数と型
2	1	クラスとオブジェクト	クラス定義、インスタンス化、メソッド呼び出し
3	1	カプセル化	private / public、getter / setter、データ保護
4	1	コンストラクタ	初期化、this、オーバーロード
5	2	継承	extends、super、メソッドオーバーライド
6	2	インターフェース	interface、implements、設計の抽象化
7	3	多態性（ポリモーフィズム）	アップキャスト、動的ディスパッチ
8	3	戦略パターン	頭脳差し替え、依存性注入
9	4	ロボットの頭脳	Brain インターフェース、AI エージェント
10	4	世界モデル	State / Action、予測と意思決定
11	5	pub/sub 通信	トピック、購読、ブロードキャスト
12	5	マルチエージェント	複数ノードの協調、同期
13	—	総合演習	ロボット犬シミュレータの完成
14	—	発表	自由課題の発表
15	—	まとめ	総復習と発展

評価基準

- 演習問題（各課末）: 40%
- 総合演習: 30%
- 自由課題発表: 30%

使用環境

- Java 21
- 任意の Java IDE（Eclipse / IntelliJ IDEA / VS Code）
- ブラウザ上の Java プレイグラウンド（TeaVM）

第0課：Java によるこそ

学習目標

- Java とは何かを説明できる
- はじめての Java プログラムを書いて実行できる
- 変数とデータ型を理解する
- プレイグラウンドでコードを動かせる

1. Java ってどんな言語？

Java は **世界でもっとも使われているプログラミング言語のひとつ** です。

- 1995 年に Sun Microsystems が開発
- 「Write Once, Run Anywhere」(一度書けばどこでも動く)
- ロボット、スマホアプリ、Web サービス、銀行システムまで幅広く使われる

なぜロボティクスに Java なのか？

- 大規模システムに向いている（ロボットは多くの部品が協調して動く）
- オブジェクト指向が言語レベルでしっかりしている
- Android のアプリは Java / Kotlin で書かれる

2. 最初のプログラム

Java のプログラムは **クラス** の中に書きます。「クラス」はまだよくわからなくて大丈夫。今は「プログラムを入れる箱」だと思ってください。

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("こんにちは、ロボット犬の世界！");  
    }  
}
```

解説

部分	意味
<code>public class Main</code>	クラス宣言。Main という名前の箱
<code>public static void main(String[] args)</code>	プログラムの入口。ここから実行が始まる
<code>System.out.println(...)</code>	画面に文字を表示する命令
<code>//</code>	コメント。プログラムの説明を書く（実行はされない）

3. プレイグラウンドで動かしてみよう

- 1. <https://bonsai.github.io/ros-kids/playground/> を開く
- 2. 左のテキストエリアに上のコードを書く
- 3. 「実行」 ボタンを押す
- 4. 右側に結果が表示される

4. 変数とデータ型

プログラムは **変数** を使ってデータを扱います。

```
public class Main {
    public static void main(String[] args) {
        // 変数の宣言と代入
        String name = "ポチ";
        int age = 3;
        double weight = 12.5;
        boolean isRobot = false;

        System.out.println("名前: " + name);
        System.out.println("年齢: " + age);
        System.out.println("体重: " + weight);
        System.out.println("ロボット?: " + isRobot);
    }
}
```

Java の主なデータ型

型	意味	例
<code>int</code>	整数	<code>42</code> , <code>-3</code> , <code>0</code>
<code>double</code>	小数	<code>3.14</code> , <code>-0.5</code>
<code>String</code>	文字列	<code>"こんにちは"</code>
<code>boolean</code>	真偽値	<code>true</code> , <code>false</code>

まとめ

- Java は「一度書けばどこでも動く」言語
- プログラムは `main` メソッドから始まる
- `System.out.println()` で画面に表示する
- 変数を使うとデータを名前で扱える

キーワード

`クラス` `main` `メソッド` `変数` `データ型` `String` `int` `boolean`

演習問題

1. 自分の名前を表示するプログラムを書いてください。
2. ロボット犬の名前・バッテリー残量（%・整数）・稼働時間（時間・小数）を変数で宣言し、表示するプログラムを書いてください。
3. `println` と `print` の違いを調べてください（ヒント：改行の有無）。

第1課：クラスとオブジェクト

学習目標

- クラスとオブジェクトの違いを説明できる
- 自分のクラスを定義できる
- クラスからオブジェクト（インスタンス）を作れる
- メソッドを呼び出せる

1. ストーリー：研究所にロボット犬が届いた

きみはロボット研究所の新人エンジニア。今日、段ボール箱が届いた。中には **ロボット犬「Aibo-X」** が入っている。

「よし、このロボット犬を動かしてみよう！」でも…何から始めればいい？

ロボット犬を動かすには、**設計図（クラス）** が必要だ。

2. クラスってなに？

クラス は **設計図** です。オブジェクト（実際のモノ）を作るための型紙のようなもの。

クラス＝設計図

オブジェクト＝設計図から作った実際のモノ（インスタンス）

```
// これは設計図（クラス）
public class RobotDog {

    // プロパティ（状態）
    String name;
    int battery;

    // メソッド（動作）
    void bark() {
        System.out.println(name + "：ワンワン！");
    }

    void walk() {
        if (battery >= 10) {
            System.out.println(name + "：歩いている...");
            battery -= 10;
        } else {
            System.out.println(name + "：バッテリー切れ！");
        }
    }
}
```

3. オブジェクトを作る

設計図ができたなら、実際に **オブジェクト（インスタンス）** を作ります。

```
public class Main {
    public static void main(String[] args) {

        // オブジェクトを作る（インスタンス化）
        RobotDog myDog = new RobotDog();

        // プロパティに値を設定
        myDog.name = "Aibo-X";
        myDog.battery = 100;

        // メソッドを呼び出す
        myDog.bark(); // Aibo-X：ワンワン！
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：歩いている...
        myDog.walk(); // Aibo-X：バッテリー切れ！
    }
}
```

インスタンス化のしくみ

```
RobotDog myDog = new RobotDog();
```

部分	意味
RobotDog	型（クラス名）
myDog	変数名
new RobotDog()	新しいオブジェクトを作る

4. 同じクラスから複数のオブジェクトを作れる

設計図が1枚あれば、何体でもロボット犬を作れます。

```
public class Main {  
    public static void main(String[] args) {  
  
        RobotDog dog1 = new RobotDog();  
        dog1.name = "Aibo-X";  
        dog1.battery = 100;  
  
        RobotDog dog2 = new RobotDog();  
        dog2.name = "ポチ";  
        dog2.battery = 80;  
  
        dog1.bark(); // Aibo-X：ワンワン！  
        dog2.bark(); // ポチ：ワンワン！  
    }  
}
```

`dog1` と `dog2` は **別々のオブジェクト** です。名前もバッテリーも独立しています。

まとめ

- クラスは設計図、オブジェクトは実際のモノ
- `new` クラス名() でオブジェクトを作る
- 同じクラスからたくさんのオブジェクトを作れる
- オブジェクトはそれぞれ独立した状態を持つ

キーワード

クラス オブジェクト インスタンス インスタンス化 `new` プロパティ メソッド

演習問題

1. ロボット犬に `charge()` メソッドを追加してください（バッテリーを満タンにする）。
2. `bark()` を呼ぶたびにバッテリーが5減るように変更してください。
3. 3体のロボット犬を作り、それぞれ別の名前をつけて吠えさせてください。

第2課：カプセル化

学習目標

- カプセル化の概念を説明できる
- `private` と `public` を使い分けられる
- getter / setter を実装できる
- カプセル化がなぜ重要なのかを説明できる

1. ストーリー：ロボット犬が壊れた

研究室で、新人エンジニアの田中くんが叫んだ。

「ロボット犬のバッテリーがマイナスになってる！」

どうやら誰かが直接 `dog.battery = -999;` と書いたらしい。

こんなことを防ぐにはどうすればいいだろう？

2. カプセル化とは

カプセル化（Encapsulation）は、「中身を隠して、外から触れるところだけ見せる」仕組みです。

カプセル化 = データ（プロパティ）を `private` で保護する
+ 必要な操作だけ `public` メソッドで公開する

カプセル化の前（悪い例）

```
class RobotDog {  
    String name;  
    int battery; // どこからでも直接書き換え可能！  
}
```

誰でも `dog.battery = -999;` とできてしまう。

カプセル化の後（良い例）

```
public class RobotDog {
    private String name;
    private int battery; // private → このクラスの中からしか触れない

    public RobotDog(String name) {
        this.name = name;
        this.battery = 100;
    }

    // getter : 値を取得するだけ
    public String getName() {
        return name;
    }

    public int getBattery() {
        return battery;
    }

    // setter : 条件つきで値を変更する
    public void charge() {
        battery = 100;
    }

    public void walk() {
        if (battery >= 10) {
            System.out.println(name + " : 歩いている...");
            battery -= 10;
        } else {
            System.out.println(name + " : バッテリー切れ！充電してください");
        }
    }
}
```

3. private と public

アクセス修飾子	アクセスできる範囲
public	どこからでも
private	同じクラスの中だけ
(指定なし)	同じパッケージの中だけ

アクセス制御のイメージ

```
public = だれでも押せるボタン（操作パネル）
private = 基板の中身、ふたを開けないと触れない
```

4. getter と setter のデザインパターン

getter：値を読み取るためのメソッド。名前の慣習は `get` プロパティ名

```
public String getName() {
    return name;
}
```

setter：値を変更するためのメソッド。名前の慣習は `set` プロパティ名

```
public void setName(String name) {
    if (name != null && name.length() > 0) {
        this.name = name;
    }
}
```

setter の中に **バリデーション（値の検証）** を入れられるのがカプセル化の利点です。

5. カプセル化のメリット

1. データの不正を防ぐ

- バッテリーがマイナスになるのを防げる

2. 内部実装を変更しやすい

- バッテリーの計算方法を変えても、外からの使い方は変わらない

3. 使い方がシンプルになる

- 「このメソッドを呼べばいい」とわかっていれば中身を知らなくてよい

まとめ

- カプセル化は「中身を隠して、必要な操作だけ公開する」
- プロパティは `private`、メソッドは原則 `public`
- getter / setter で安全にデータを出入りさせる

- カプセル化によってプログラムは壊れにくくなる

キーワード

カプセル化 private public アクセス修飾子 getter setter バリデーション

演習問題

1. ロボット犬の `battery` が 0～100 の範囲を超えないように setter を実装してください。
2. ロボット犬に `run()`（走る）メソッドを追加してください。走るとバッテリーが 25 減ります。
3. 「バッテリー残量が 20 を切ったら充電を促す」機能を `walk()` に追加してください。

第3課：コンストラクタと初期化

学習目標

- コンストラクタを定義できる
- `this` キーワードの役割を説明できる
- コンストラクタのオーバーロードができる
- 初期化のベストプラクティスを理解する

1. ストーリー：ロボット犬を組み立てよう

研究所に新しいロボット犬のキットが届いた。組み立てるときに、最初から名前とバッテリーを設定したい。

前回までは、オブジェクトを作ってからプロパティを設定していた。

```
RobotDog dog = new RobotDog();  
dog.name = "Aibo-X";    // あとから設定  
dog.battery = 100;      // あとから設定
```

「オブジェクトを作るときに、いっしょに初期設定できないかな？」

2. コンストラクタ

コンストラクタ は、オブジェクトが作られるときに自動的に呼ばれる特殊なメソッドです。

```
public class RobotDog {
    private String name;
    private int battery;

    // コンストラクタ（クラス名と同じ名前）
    public RobotDog(String name, int battery) {
        this.name = name;
        this.battery = battery;
    }

    public RobotDog(String name) {
        this.name = name;
        this.battery = 100; // デフォルト値
    }
}
```

使い方

```
public class Main {
    public static void main(String[] args) {

        RobotDog dog1 = new RobotDog("Aibo-X", 80); // 名前とバッテリーを指定
        RobotDog dog2 = new RobotDog("ポチ");       // 名前だけ指定（バッテリーはデフォルト100）
    }
}
```

3. this キーワード

`this` は「自分自身のオブジェクト」を指します。

```
public RobotDog(String name, int battery) {
    this.name = name; // this.name = プロパティ, name = 引数
    this.battery = battery;
}
```

これがないと、プロパティと引数が同じ名前のとき区別が付きません。

4. コンストラクタのオーバーロード

同じ名前でも、引数の種類や数が違うコンストラクタを複数定義できます。これを **オーバーロード (overload)** といいます。

```
public class RobotDog {

    private String name;
    private int battery;
    private String model;

    // ① 全部指定
    public RobotDog(String name, int battery, String model) {
        this.name = name;
        this.battery = battery;
        this.model = model;
    }

    // ② 名前とモデルだけ（バッテリーはデフォルト）
    public RobotDog(String name, String model) {
        this(name, 100, model); // ①を呼び出す
    }

    // ③ 名前だけ（モデルは"Standard"）
    public RobotDog(String name) {
        this(name, 100, "Standard"); // ①を呼び出す
    }

    // ④ 引数なし（すべてデフォルト）
    public RobotDog() {
        this("名無しロボット", 100, "Standard"); // ①を呼び出す
    }
}
```

使い方

```
public class Main {
    public static void main(String[] args) {

        RobotDog d1 = new RobotDog("Aibo-X", 80, "RX-1000");
        RobotDog d2 = new RobotDog("ポチ", "RX-500");
        RobotDog d3 = new RobotDog("タロウ");
        RobotDog d4 = new RobotDog();

    }
}
```

5. デフォルトコンストラクタ

コンストラクタを1つも書かないと、Java が自動的に **デフォルトコンストラクタ** を用意します。

```
public RobotDog() {  
    // 何もしない（プロパティはデフォルト値）  
}
```

ただし、自分でコンストラクタを書くと、デフォルトコンストラクタは自動生成されなくなります。

まとめ

- コンストラクタはオブジェクト生成時に自動実行される
- `this` は自分自身を指す
- オーバーロードで複数の初期化パターンを提供できる
- `this(...)` で別のコンストラクタを呼べる

キーワード

コンストラクタ `this` オーバーロード デフォルトコンストラクタ 初期化

演習問題

1. ロボット犬に `color`（色）プロパティを追加し、コンストラクタで設定できるようにしてください。
2. デフォルトコンストラクタで「名無しロボット・白・バッテリー50」をデフォルト値にするコードを書いてください。
3. コンストラクタ内でバッテリーの値が0～100の範囲であることをチェックし、範囲外なら例外を投げる処理を追加してください。

第4課：継承

学習目標

- 継承の概念を説明できる
- `extends` を使ってクラスを継承できる
- メソッドのオーバーライドができる
- `super` で親クラスのメソッドを呼べる
- 「is-a」関係を理解する

1. ストーリー：スーパーロボット犬の開発

研究所で新しいプロジェクトが始まった。「Aibo-X をベースに、もっとすごいロボット犬を作ろう！」

でも、ゼロから作るのは大変…。 そうだ、**既存のロボット犬の機能を引き継いで、新しい機能を追加しよう。**

2. 継承とは

継承 (Inheritance) は、既存のクラス（親クラス）の機能を引き継いで、新しいクラス（子クラス）を作る仕組みです。

```
親クラス（スーパークラス） = 基本的な機能
    ↓ extends
子クラス（サブクラス）      = 基本機能 + 独自機能
```

ロボット犬の継承

```
// 親クラス
public class RobotDog {
    protected String name;
    protected int battery;

    public RobotDog(String name) {
        this.name = name;
        this.battery = 100;
    }

    public void bark() {
        System.out.println(name + "：ワンワン！（標準音声）");
    }

    public void walk() {
        if (battery >= 10) {
            System.out.println(name + "：歩いている...");
            battery -= 10;
        }
    }

    public void charge() {
        battery = 100;
        System.out.println(name + "：充電完了");
    }
}

// 子クラス（スーパーロボット犬）
public class SuperRobotDog extends RobotDog {

    private boolean flyMode;

    public SuperRobotDog(String name) {
        super(name); // 親クラスのコンストラクタを呼ぶ
        this.flyMode = false;
    }

    // 新しい機能：飛ぶ
    public void fly() {
        if (battery >= 30) {
            System.out.println(name + "：飛行モード起動！");
            battery -= 30;
            flyMode = true;
        } else {
            System.out.println(name + "：バッテリー不足で飛行できません");
        }
    }

    // 親のメソッドをオーバーライド（上書き）
    @Override
    public void bark() {
```

```
        System.out.println(name + "：スーパーワンワン！（大音量）");
    }
}
```

3. extends キーワード

`extends` で親クラスを指定します。

```
public class SuperRobotDog extends RobotDog {
    // RobotDog のすべての機能 + 追加機能
}
```

- 子クラスは親クラスの `public` / `protected` なメンバーを使える
- `private` なメンバーは継承されない

4. メソッドのオーバーライド

オーバーライド（`override`）は、親クラスのメソッドを子クラスで上書きすることです。

```
@Override // ← コンパイラに「オーバーライドしてるよ」と教える印
public void bark() {
    System.out.println(name + "：スーパーワンワン！（大音量）");
}
```

`@Override` をつけると、間違って親クラスにないメソッド名を書いたときに コンパイルエラーになって教えてくれます。

5. super キーワード

`super` は親クラスを指します。

```
public SuperRobotDog(String name) {
    super(name); // 親クラスのコンストラクタを呼ぶ
}

public void charge() {
    super.charge(); // 親のcharge()を呼ぶ
    System.out.println("＋高速充電モード");
}
```

6. 継承のメリット

1. コードの再利用

- 共通の機能は親クラスに一度書けばいい

2. 階層構造を作れる

- RobotDog → SuperRobotDog → FlyingRobotDog のように拡張できる

3. 変更に強い

- 親クラスを修正すると、すべての子クラスに反映される

7. protected アクセス修飾子

修飾子	同じクラス	同じパッケージ	子クラス	どこからでも
private	○	×	×	×
(なし)	○	○	×	×
protected	○	○	○	×
public	○	○	○	○

子クラスから親のプロパティにアクセスしたいときは `protected` を使います。

まとめ

- 継承は「既存のクラスをベースに新しいクラスを作る」
- `extends` で親クラスを指定する
- `@Override` でメソッドを上書きできる
- `super` で親クラスのメソッドを呼べる
- 「is-a」関係（スーパーロボット犬はロボット犬の一種）を表す

キーワード

継承 extends スーパークラス サブクラス オーバーライド @Override super protected

演習問題

1. `FlyingRobotDog` クラスを作成し、`SuperRobotDog` を継承して `hover()` (ホバリング) メソッドを追加してください。
2. `bark()` をオーバーライドして、飛行中は「フライト中は吠えられません」と表示するようにしてください。
3. 親クラスの `walk()` をオーバーライドして、飛行中は消費バッテリーが半分になるようにしてください。

第5課：インターフェースによる抽象化

学習目標

- インターフェースの概念を説明できる
- `interface` を定義できる
- `implements` でインターフェースを実装できる
- 継承とインターフェースの違いを説明できる

1. ストーリー：リアル犬とロボット犬を同じに扱いたい

研究所には「Aibo-X（ロボット犬）」と「ポチ（本物の犬）」がいる。

2匹はまったく違う——

- ポチは生物で、Aibo-X は機械
- ポチはご飯を食べ、Aibo-X は充電する
- ポチは散歩が必要、Aibo-X はプログラムで歩く

でも、どちらも「犬」として **同じ操作で扱いたい**。

ここで **インターフェース** が役立つ。

2. インターフェースとは

インターフェース は「こんなメソッドを持っていること」を約束するものです。メソッドの「名前と引数と戻り値」だけを定義し、中身は書きません。

```
// インターフェース：犬としての約束事
public interface Dog {
    void bark();
    void walk();
    void play();
}
```

インターフェースの中のメソッドは、すべて **抽象メソッド**（中身のないメソッド）です。

3. implements で実装する

インターフェースの「約束」を守るには `implements` を使います。

```
public class RealDog implements Dog {  
  
    private String name;  
  
    public RealDog(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void bark() {  
        System.out.println(name + "：ワンワン！（本物の吠え声）");  
    }  
  
    @Override  
    public void walk() {  
        System.out.println(name + "：とことこ散歩している...");  
    }  
  
    @Override  
    public void play() {  
        System.out.println(name + "：ボールを追いかけている！");  
    }  
}
```

```
public class RobotDog implements Dog {

    private String name;

    public RobotDog(String name) {
        this.name = name;
    }

    @Override
    public void bark() {
        System.out.println(name + "：ピコーン！吠え音声再生");
    }

    @Override
    public void walk() {
        System.out.println(name + "：モーター駆動で歩行中...");
    }

    @Override
    public void play() {
        System.out.println(name + "：プログラムされた遊び動作を実行");
    }

}
```

4. 多態性（ポリモーフィズム）への準備

インターフェースを使うと、異なるクラスを **同じ型** として扱えます。

```
public class Main {
    public static void main(String[] args) {

        Dog[] dogs = {
            new RealDog("ポチ"),
            new RobotDog("Aibo-X")
        };

        for (Dog d : dogs) {
            d.bark(); // 同じ bark() でも中身が違う！
            d.walk();
            d.play();
            System.out.println("---");
        }
    }
}
```

これが **多態性** への第一歩です。（多態性は第6課で詳しく学びます）

5. 複数のインターフェースを実装する

Java のクラスは 1つのクラスしか継承できないが、インターフェースは複数実装できる。

```
public interface Charger {
    void charge();
}

public interface GPS {
    double[] getLocation();
}

public class SuperRobotDog implements Dog, Charger, GPS {
    // Dog のメソッド + charge() + getLocation() をすべて実装する
}
```

6. 継承 vs インターフェース

	継承 (extends)	インターフェース (implements)
意味	「is-a」関係	「〜できる」能力
中身	実装を持てる	抽象メソッドのみ
複数	1つだけ	複数可能
例	RobotDog → SuperRobotDog	Dog, Charger, GPS

使い分けの指針

- **継承**：機能の再利用が目的。親子関係が明確なとき。
- **インターフェース**：共通の操作を定義するのが目的。全く違うクラスを同じように扱いたいとき。

まとめ

- インターフェースは「メソッドの約束事」を定義する
- `implements` でインターフェースを実装する
- 異なるクラスも同じインターフェース型として扱える
- 複数のインターフェースを同時に実装できる
- 継承は「is-a」、インターフェースは「〜できる」

キーワード

インターフェース implements 抽象メソッド 多態性 複数実装 能力

演習問題

1. `Swimmable` インターフェースを作成し（`swim()` メソッド）、RobotDog に実装してください。
2. `Sensor` インターフェースを作成し（`double readDistance()`）、RobotDog に実装して距離を返すようにしてください。
3. RealDog と RobotDog に、共通のインターフェースを通して「お手」をさせるプログラムを書いてください。

第6課：多態性（ポリモーフィズム）

学習目標

- 多態性の概念を説明できる
- アップキャストとダウンキャストを理解する
- 動的ディスパッチのしくみを説明できる
- 多態性がコードをどうシンプルにするかを説明できる

1. ストーリー：同じリモコンで全部の犬を操作したい

研究所の机の上に、たくさんのリモコンが散らばっている。

「ポチ用リモコン」「Aibo-X 用リモコン」「スーパーロボット犬用リモコン」

「めんどくさい！どれも『犬』なんだから、1つのリモコンで全部操作したい！」

これこそが **多態性** が解決する問題。

2. 多態性とは

多態性（ポリモーフィズム / polymorphism） は、「同じ操作でも、相手のクラスによって振る舞いが変わる」仕組みです。

polymorphism = poly（多くの） + morph（形） = 多くの形

アナロジー：「再生ボタン」

アプリ	ボタンを押す	結果
音楽アプリ	再生ボタン	音楽が流れる
動画アプリ	再生ボタン	動画が再生される
録音アプリ	再生ボタン	録音が再生される

ボタンは同じ「再生」なのに、中身の動きはアプリによって違う。これが多態性。

3. コードで見る多態性

```
public interface Dog {
    void bark();
    void walk();
    void play();
}

public class RealDog implements Dog {
    public void bark() { System.out.println("ワンワン!"); }
    public void walk() { System.out.println("とことこ歩く"); }
    public void play() { System.out.println("ボールで遊ぶ"); }
}

public class RobotDog implements Dog {
    public void bark() { System.out.println("ピコーン！吠え音再生"); }
    public void walk() { System.out.println("モーター駆動で歩行"); }
    public void play() { System.out.println("プログラムされた遊び動作"); }
}
```

多態性の力

```
public class Main {
    public static void main(String[] args) {

        Dog[] dogs = {
            new RealDog(),
            new RobotDog()
        };

        // 呼び出す側は「何の犬か」を気にしていない！
        for (Dog d : dogs) {
            d.bark(); // 勝手に正しい吠え方になる
            d.walk(); // 勝手に正しい歩き方になる
        }
    }
}
```

呼び出す側（for 文）は Dog 型としか向き合っていない。それなのに、実際のオブジェクトに応じて正しい動作が選ばれる。

4. アップキャストと動的ディスパッチ

アップキャスト

アップキャスト は、子クラスのオブジェクトを親クラス（またはインターフェース）型の変数に代入すること。

```
Dog dog = new RealDog(); // アップキャスト (RealDog → Dog)
```

「具体的な型を、より抽象的な型として扱う」こと。

動的ディスパッチ

動的ディスパッチ (dynamic dispatch) は、**実行時に** どのメソッドを呼ぶかが決まる仕組み。

```
Dog dog;

if (isRealDog) {
    dog = new RealDog();
} else {
    dog = new RobotDog();
}

dog.bark(); // 実行時に isRealDog の値によって呼ばれる bark() が変わる
```

コンパイル時には決まらず、**実行時のオブジェクトの中身** で動作が決まる。

5. 多態性がなかった世界（比較）

多態性がない場合（if 文地獄）

```
void makeSound(Object animal) {
    if (animal instanceof RealDog) {
        ((RealDog) animal).bark();
    } else if (animal instanceof RobotDog) {
        ((RobotDog) animal).bark();
    } else if (animal instanceof SuperRobotDog) {
        ((SuperRobotDog) animal).bark();
    }
    // 新しい犬の種類が増えるたびにここを書き換える
}
```

多態性がある場合

```
void makeSound(Dog dog) {  
    dog.bark(); // 新しい犬が増えても修正不要!  
}
```

6. 多態性の3つの条件

1. 継承またはインターフェース実装 があること
2. メソッドがオーバーライド されていること
3. 親クラス（インターフェース）型の変数 で参照すること

この3つがそろると、多態性が発動します。

まとめ

- 多態性 = 同じ操作でも相手によって動きが変わる
- アップキャスト：具体的な型 → 抽象的な型
- 動的ディスパッチ：実行時にメソッドが決まる
- 多態性で条件分岐が減り、コードがシンプルになる
- 多態性の3条件：継承/実装 + オーバーライド + 親クラス型

キーワード

多態性 ポリモーフィズム アップキャスト 動的ディスパッチ instanceof オーバーライド

演習問題

1. `Dog` インターフェースに `eat()` メソッドを追加し、`RealDog` と `RobotDog` で異なる実装をしてください。
2. 多態性を使わずに `if-else` で動物の種類を判別するコードを書き、その問題点を説明してください。
3. `List<Dog>` を使って 3 種類以上の犬を追加し、それぞれの `bark()` を一度のループで呼び出すコードを書いてください。

第7課：戦略パターン——頭脳の差し替え

学習目標

- 戦略パターン（Strategy Pattern）を理解する
- 多態性を使って「頭脳の差し替え」を実装できる
- 依存性注入（Dependency Injection）の初歩を理解する
- 「継承よりコンポジション」の意味を説明できる

1. ストーリー：ロボット犬の頭脳を交換したい

研究所で、ロボット犬の「性格」を変えたいという要望が出た。

- 探索が好きな「好奇心旺盛モード」
- 慎重に動く「安全第一モード」
- ランダムに動く「ランダムモード」

でも、頭脳を変えるたびにクラスをまるごと書き換えるのは大変。「頭脳（Brain）」だけを差し替えられるようにしたい。

2. 戦略パターンとは

戦略パターン（Strategy Pattern） は、アルゴリズム（戦略）をクラスとしてカプセル化し、実行時に差し替えられるようにするデザインパターン。

エージェント（身体） → 固定
戦略（頭脳） → 差し替え可能

3. 頭脳を差し替えるコード

Brain インターフェース

```
public interface Brain {  
    String decide(String situation);  
}
```

3 種類の頭脳

```
public class CuriousBrain implements Brain {  
    @Override  
    public String decide(String situation) {  
        return "未知の場所を探索しに行く！";  
    }  
}  
  
public class CautiousBrain implements Brain {  
    @Override  
    public String decide(String situation) {  
        return "周囲を確認してから安全なルートを進む";  
    }  
}  
  
public class RandomBrain implements Brain {  
    @Override  
    public String decide(String situation) {  
        double r = Math.random();  
        if (r < 0.33) return "左に行く";  
        else if (r < 0.66) return "右に行く";  
        else return "その場で待機する";  
    }  
}
```

エージェント（頭脳を持ったロボット犬）

```
public class RobotDogAgent {

    private String name;
    private Brain brain; // 頭脳 (差し替え可能)

    public RobotDogAgent(String name, Brain brain) {
        this.name = name;
        this.brain = brain;
    }

    // 頭脳を後から差し替え
    public void setBrain(Brain brain) {
        this.brain = brain;
    }

    public void act(String situation) {
        System.out.println(name + " は " + situation + " で " +
                           brain.decide(situation));
    }
}
```

使ってみる

```
public class Main {
    public static void main(String[] args) {

        RobotDogAgent agent = new RobotDogAgent("Aibo-X", new CuriousBrain());

        agent.act("見知らぬ部屋にいる"); // 探索に行く

        // 頭脳を差し替え！
        agent.setBrain(new CautiousBrain());
        agent.act("見知らぬ部屋にいる"); // 安全ルートを進む

        // さらに差し替え！
        agent.setBrain(new RandomBrain());
        agent.act("見知らぬ部屋にいる"); // ランダムな行動
    }
}
```

実行結果

```
Aibo-X は 見知らぬ部屋にいる で 未知の場所を探索しに行く！
Aibo-X は 見知らぬ部屋にいる で 周囲を確認してから安全なルートを進む
Aibo-X は 見知らぬ部屋にいる で 右に行く
```

4. 依存性注入（Dependency Injection）

上のコードでは、RobotDogAgent のコンストラクタで Brain を受け取っている。これが **依存性注入（DI）** の初歩的な形。

```
// NG：エージェントが自分で頭脳を作る（密結合）
public class RobotDogAgent {
    private Brain brain = new CuriousBrain(); // ← 差し替えにくい
}

// OK：外から頭脳を受け取る（疎結合）
public class RobotDogAgent {
    private Brain brain;
    public RobotDogAgent(Brain brain) {
        this.brain = brain; // ← 外から注入される
    }
}
```

5. 継承よりコンポジション

「継承よりコンポジション（composition over inheritance）」は、「機能を追加するときは、継承より部品を組み合わせるほうが柔軟」という設計原則。

方法	柔軟性	例
継承	低い（親子関係が固定）	SuperRobotDog extends RobotDog
コンポジション	高い（部品を差し替え可能）	RobotDogAgent has-a Brain

AI エージェントの頭脳差し替えは、この原則の代表例。

まとめ

- 戦略パターンは「アルゴリズムを部品として差し替え可能にする」
- Brain インターフェースで頭脳を抽象化する
- 依存性注入で外部から部品を受け取る
- 「継承よりコンポジション」で柔軟な設計になる
- これは多態性の実践的な応用

キーワード

演習問題

1. 新しい `SleepyBrain` クラスを作成し、常に「眠いから寝る」と返すようにしてください。
2. 状況によって頭脳を自動的に切り替える `AdaptiveBrain` を作成してください（例：バッテリーが少ないときは `CautiousBrain` に切り替える）。
3. `RobotDogAgent` に `battery` プロパティを追加し、行動ごとに消費する仕組みを実装してください。

第8課：ロボットの頭脳を入れ替える

学習目標

- AI エージェントの基本構造を理解する
- センサー入力 → 頭脳 → 行動出力の流れを実装できる
- 複数の頭脳を切り替えながら実験できる

1. ストーリー：ロボット犬の頭脳研究所

研究所のメインプロジェクトは「頭脳の研究」。

ロボット犬のハードウェアは同じでも、**頭脳（アルゴリズム）**を変えるだけでまったく違う振る舞いになる。

これは「世界モデル（World Model）」への第一歩。

2. AI エージェントの基本構造

AI エージェントは以下の3つの要素からなる。

センサー入力 → 頭脳（意思決定） → アクチュエータ（行動）

ロボット犬で言い換えると：

カメラ・マイク → `Brain.decide()` → モーター・スピーカー

3. 完全なエージェントの実装

状態（State）

```
public class State {  
    private int battery;  
    private boolean hasBall;  
    private String location;  
  
    public State(int battery, boolean hasBall, String location) {  
        this.battery = battery;  
        this.hasBall = hasBall;  
        this.location = location;  
    }  
  
    public int getBattery() { return battery; }  
    public boolean hasBall() { return hasBall; }  
    public String getLocation() { return location; }  
  
    public State withBattery(int battery) {  
        return new State(battery, hasBall, location);  
    }  
  
    @Override  
    public String toString() {  
        return "State{battery=" + battery + ", hasBall=" + hasBall +  
            ", location='" + location + "'}";  
    }  
}
```

行動 (Action)

```
public abstract class Action {
    private String name;
    public Action(String name) { this.name = name; }
    public String getName() { return name; }
    public abstract State execute(State current);
}

public class WalkAction extends Action {
    public WalkAction() { super("歩<"); }
    @Override
    public State execute(State current) {
        return current.withBattery(current.getBattery() - 10);
    }
}

public class PlayAction extends Action {
    public PlayAction() { super("遊ぶ"); }
    @Override
    public State execute(State current) {
        return current.withBattery(current.getBattery() - 5);
    }
}

public class ChargeAction extends Action {
    public ChargeAction() { super("充電する"); }
    @Override
    public State execute(State current) {
        return current.withBattery(100);
    }
}

public class SearchAction extends Action {
    public SearchAction() { super("探索する"); }
    @Override
    public State execute(State current) {
        return current.withBattery(current.getBattery() - 15);
    }
}
```

頭脳 (Brain) インターフェース

```
public interface Brain {
    Action decide(State currentState);
}
```

頭脳の実装：SimpleBrain

```
public class SimpleBrain implements Brain {
    @Override
    public Action decide(State state) {
        if (state.getBattery() < 20) {
            return new ChargeAction();
        }
        if (state.hasBall()) {
            return new PlayAction();
        }
        return new SearchAction();
    }
}
```

エージェント本体

```
public class RobotDogAgent {

    private String name;
    private Brain brain;
    private State state;

    public RobotDogAgent(String name, Brain brain) {
        this.name = name;
        this.brain = brain;
        this.state = new State(100, false, "研究所");
    }

    public void setBrain(Brain brain) {
        this.brain = brain;
        System.out.println(name + " : 頭脳を交換しました");
    }

    public void step() {
        System.out.println("現在の状態：" + state);
        Action action = brain.decide(state);
        System.out.println(name + " の選択：" + action.getName());
        state = action.execute(state);
        System.out.println("実行後の状態：" + state);
        System.out.println("---");
    }
}
```

実行

```
public class Main {
    public static void main(String[] args) {

        RobotDogAgent agent = new RobotDogAgent("Aibo-X", new SimpleBrain());

        for (int i = 0; i < 5; i++) {
            agent.step();
        }
    }
}
```

実行結果の例

```
現在の状態：State{battery=100, hasBall=false, location='研究所'}
Aibo-X の選択：探索する
実行後の状態：State{battery=85, hasBall=false, location='研究所'}
---
現在の状態：State{battery=85, hasBall=false, location='研究所'}
Aibo-X の選択：探索する
実行後の状態：State{battery=70, hasBall=false, location='研究所'}
---
... (繰り返し) ...
現在の状態：State{battery=15, hasBall=false, location='研究所'}
Aibo-X の選択：充電する
実行後の状態：State{battery=100, hasBall=false, location='研究所'}
---
```

4. 頭脳の差し替え実験

```
public class Main {
    public static void main(String[] args) {

        RobotDogAgent agent = new RobotDogAgent("Aibo-X", new SimpleBrain());

        // シンプル頭脳で3ステップ
        System.out.println("=== SimpleBrain ===");
        for (int i = 0; i < 3; i++) agent.step();

        // ランダム頭脳に差し替え
        agent.setBrain(new RandomBrain());
        System.out.println("=== RandomBrain ===");
        for (int i = 0; i < 3; i++) agent.step();
    }
}
```

5. これが世界モデルへの入り口

この「状態 → 判断 → 行動 → 状態変化」のループは、**世界モデル (World Model)** の最も単純な形。

- 状態 (State) = 世界の今の状況
- 行動 (Action) = ロボットの動作
- 頭脳 (Brain) = 状態から最適な行動を選ぶルール

第9課では、これを「予測」に拡張する。

まとめ

- AI エージェントは「センサー → 頭脳 → 行動」で構成される
- State で世界の状態を表現する
- Action でロボットの行動を表現する
- Brain が状態から最適な行動を選ぶ
- 頭脳を差し替えるだけで振る舞いが変わる

キーワード

AI エージェント 状態 行動 センサー アクチュエータ 意思決定 世界モデル

演習問題

1. 新しい頭脳 `AggressiveBrain` (バッテリーが 50 以上なら常に探索、それ以下なら充電) を作成してください。
2. State に `temperature` (温度) プロパティを追加し、温度が高いときは充電を控える頭脳を作成してください。
3. 頭脳が選んだ行動の履歴を保持する `LoggingBrain` (実際の頭脳をラップする) を作成してください。

第9課：世界モデル——状態と行動の予測

学習目標

- 世界モデル（World Model）の概念を説明できる
- 状態遷移の予測をコード化できる
- 「予測 → 選択 → 実行」のループを実装できる
- 多態性と世界モデルの関係を説明できる

1. ストーリー：未来を予測するロボット犬

これまでのロボット犬は「今の状態だけ」で行動を選んでいた。でも、本当に賢いロボットは「この行動をしたら、次はどうなるか」を予測して行動を選ぶ。

今の Aibo-X の問題：

```
バッテリー残量：25
↓
バッテリーが少ないから「充電する」を選ぶ
↓
つまらない...
```

「充電したらその後どうなる？」を予測できれば、「充電してから探索に行こう」という計画を立てられる。

2. 世界モデルとは

世界モデル（World Model）は、「行動すると世界（状態）がどう変わるか」を予測する仕組み。

```
現在の状態 + 行動 → 予測される未来の状態
```

3. 世界モデルをコード化する

世界モデルインターフェース

```
public interface WorldModel {  
    State predict(State current, Action action);  
    double evaluate(State predictedState);  
}
```

シンプルな世界モデルの実装

```
public class SimpleWorldModel implements WorldModel {  
  
    @Override  
    public State predict(State current, Action action) {  
        // 行動を仮実行して結果を予測  
        return action.execute(current);  
    }  
  
    @Override  
    public double evaluate(State predictedState) {  
        double score = 0;  
        score += predictedState.getBattery() * 0.5;           // バッテリーは高いほうが良い  
        score += predictedState.hasBall() ? 30 : 0;           // ボールがあれば嬉しい  
        score += predictedState.getBattery() >= 50 ? 20 : -20; // 50% 以上を維持したい  
        return score;  
    }  
}
```

予測する頭脳（世界モデル搭載）

```
public class PredictiveBrain implements Brain {

    private WorldModel model;
    private List<Action> possibleActions;

    public PredictiveBrain(WorldModel model) {
        this.model = model;
        this.possibleActions = Arrays.asList(
            new WalkAction(),
            new PlayAction(),
            new ChargeAction(),
            new SearchAction()
        );
    }

    @Override
    public Action decide(State currentState) {
        Action bestAction = null;
        double bestScore = Double.NEGATIVE_INFINITY;

        System.out.println("【世界モデルによる予測】");
        for (Action action : possibleActions) {
            State predicted = model.predict(currentState, action);
            double score = model.evaluate(predicted);

            System.out.println(action.getName() + " → " + predicted + " スコア:" + score);

            if (score > bestScore) {
                bestScore = score;
                bestAction = action;
            }
        }

        System.out.println("選択: " + bestAction.getName());
        return bestAction;
    }
}
```

使ってみる

```
public class Main {
    public static void main(String[] args) {

        WorldModel model = new SimpleWorldModel();
        RobotDogAgent agent = new RobotDogAgent("Aibo-X", new PredictiveBrain(model));

        // バッテリー少ない状態でスタート
        for (int i = 0; i < 4; i++) {
            agent.step();
        }
    }
}
```

実行結果（イメージ）

【世界モデルによる予測】
歩く → State{battery=15} スコア:7.5
遊ぶ → State{battery=20} スコア:10.0
充電する → State{battery=100} スコア:70.0
探索する → State{battery=10} スコア:5.0
選択: 充電する

充電後の状態: State{battery=100}

【世界モデルによる予測】
歩く → State{battery=90} スコア:65.0
遊ぶ → State{battery=95} スコア:67.5
充電する → State{battery=100} スコア:70.0
探索する → State{battery=85} スコア:62.5
選択: 充電する（または遊ぶ）

4. 多態性と世界モデルの関係

対応関係

OOP の多態性	ロボティクスの世界モデル
<code>speak()</code> の実装がクラスごとに違う	<code>grasp()</code> の動作が物体ごとに違う
呼び出す側は相手の型を意識しない	制御側は物体の種類を意識しない
抽象化されたインターフェース	抽象化された行動プリミティブ
実装の違いを隠蔽	物理的な違いをモデルが吸収

本質

```
// 多態性：同じメソッド名、違う実装
animals.forEach(animal -> animal.speak());

// 世界モデル：同じコマンド、違う動作
objects.forEach(obj -> robot.grasp(obj));
```

どちらも「抽象化された同じ操作が、具体的な対象に応じて変化する」という本質を持つ。

まとめ

- 世界モデルは「行動の結果を予測する」仕組み
- `predict()` で未来の状態を予測
- `evaluate()` で予測された状態を評価
- 最適な行動を選択できる
- 世界モデルは多態性の「現実世界版」

キーワード

世界モデル World Model 予測 評価 状態遷移 意思決定 計画

演習問題

1. `evaluate()` に「バッテリーが高いほど良い」だけでなく「変動が少ないほど良い」という評価軸を追加してください。
2. 2ステップ先まで予測する頭脳 `TwoStepPredictiveBrain` を作成してください。
3. ロボット犬に「お手」と「ふせ」の Action を追加し、それらを含めて世界モデルで評価するコードを書いてください。

第10課：publish / subscribe 通信

学習目標

- ROS のノードとトピックの概念を理解する
- publish（送信）と subscribe（受信）を実装できる
- インターフェースを使って疎結合な通信を設計できる

1. ストーリー：ロボット犬どうしが会話したい

研究所にロボット犬が増えてきた。Aibo-X、ポチ、そして新しい仲間「タロウ」。

でも、それぞれがバラバラに動いている。「Aibo-X が吠えたことを、みんなに知らせたい！」

ここで **ROS（Robot Operating System）** の考え方が役立つ。

2. ROS とは

ROS（Robot Operating System）は、ロボットソフトウェアを開発するためのフレームワーク。

ROS の基本的な考え方：

```
ノード（Node） = ロボットの中のひとつの機能  
トピック（Topic） = メッセージの通り道  
publish = メッセージを送る  
subscribe = メッセージを受け取る
```

3. Java で pub/sub を実装する

購読者（Subscriber）インターフェース

```
public interface Subscriber {  
    void onMessage(String topicName, String message);  
}
```

トピック (Topic) インターフェース

```
import java.util.ArrayList;
import java.util.List;

public interface Topic {
    void publish(String message);
    void subscribe(Subscriber sub);
}
```

ブロードキャストトピック

```
public class BroadcastTopic implements Topic {

    private List<Subscriber> subs = new ArrayList<>();
    private String name;

    public BroadcastTopic(String name) {
        this.name = name;
    }

    @Override
    public void publish(String message) {
        System.out.println "[" + name + "] publish: " + message);
        for (Subscriber sub : subs) {
            sub.onMessage(name, message);
        }
    }

    @Override
    public void subscribe(Subscriber sub) {
        subs.add(sub);
    }
}
```

表示ノード

```
public class DisplayNode implements Subscriber {

    private String name;

    public DisplayNode(String name) {
        this.name = name;
    }

    @Override
    public void onMessage(String topic, String message) {
        System.out.println(" [" + name + " 表示] " + topic + " → " + message);
    }
}
```

犬ノード：自分も購読する

```
public class DogNode implements Subscriber {

    private String name;
    private Topic barkTopic;
    private Topic walkTopic;

    public DogNode(String name, Topic barkTopic, Topic walkTopic) {
        this.name = name;
        this.barkTopic = barkTopic;
        this.walkTopic = walkTopic;
    }

    public void bark() {
        barkTopic.publish(name + " : ワンワン!");
    }

    public void walk() {
        walkTopic.publish(name + " : とことこ歩く");
    }

    @Override
    public void onMessage(String topic, String message) {
        if (!message.contains(name)) {
            System.out.println(" [" + name + " 聞いた] " + topic +
                               "で「" + message + "」と言っている");
        }
    }
}
```

メイン

```

public class Main {
    public static void main(String[] args) {

        Topic barkTopic = new BroadcastTopic("bark");
        Topic walkTopic = new BroadcastTopic("walk");

        DogNode pochi = new DogNode("ポチ", barkTopic, walkTopic);
        DogNode aiboX = new DogNode("Aibo-X", barkTopic, walkTopic);

        // トピックに購読者を登録
        barkTopic.subscribe(new DisplayNode("モニター"));
        barkTopic.subscribe(pochi);
        barkTopic.subscribe(aiboX);
        walkTopic.subscribe(pochi);
        walkTopic.subscribe(aiboX);

        System.out.println("=== ポチが吠える ===");
        pochi.bark();

        System.out.println("=== Aibo-X が歩く ===");
        aiboX.walk();
    }
}

```

実行結果

```

=== ポチが吠える ===
[bark] publish: ポチ：ワンワン！
[モニター 表示] bark → ポチ：ワンワン！
[Aibo-X 聞いた] barkで「ポチ：ワンワン！」と言っている
=== Aibo-X が歩く ===
[walk] publish: Aibo-X：とことこ歩く
[ポチ 聞いた] walkで「Aibo-X：とことこ歩く」と言っている

```

4. なぜ publish / subscribe が重要なのか

1. ノード同士が直接つながっていない（疎結合）

- ポチは Aibo-X の存在を知らなくてよい
- トピックを通して間接的に通信する

2. 新しいノードを追加しやすい

- DisplayNode を後から追加しても既存のコードは修正不要

3. スケールする

- 10 匹でも 100 匹でも同じ仕組みで通信できる

まとめ

- ノードは独立した機能、トピックは通信路
- publish = メッセージを送る、subscribe = 受け取る
- pub/sub によってノード同士は疎結合になる
- 新しいノードの追加が容易になる
- これは ROS の中心的な設計思想

キーワード

ROS ノード トピック publish subscribe 疎結合 ブロードキャスト

演習問題

1. play トピックを追加し、犬が遊ぶときもメッセージを publish するようにしてください。
2. LoggerNode (すべてのトピックを購読してログに記録するノード) を作成してください。
3. 特定のメッセージだけフィルタリングする FilterNode を作成してください (例: 「ワンワン」という文字列が含まれるメッセージだけを表示する)。

第11課：マルチエージェントと協調動作

学習目標

- マルチエージェントシステムの基本を理解する
- 複数のノードが協調して動作する仕組みを実装できる
- メッセージを使ってエージェント間で情報を共有できる
- 分散システムの考え方を理解する

1. ストーリー：3 匹のロボット犬で協力しよう

研究所に 3 匹のロボット犬がいる。

- **ポチ**（探索担当）：新しい場所を探す
- **Aibo-X**（運搬担当）：見つけたものを運ぶ
- **タロウ**（警備担当）：危険を知らせる

この 3 匹が協力すれば、1 匹ではできないことができる。

2. マルチエージェントシステム

マルチエージェントシステム（MAS） は、複数の独立したエージェントが協調して問題を解決するシステム。

ロボティクスでは、複数のロボットが協力するときに使われる。

メッセージの設計

まず、犬同士がやりとりするメッセージの型を定義する。

```
public class DogMessage {
    private String sender;
    private String type;    // "bark", "found_ball", "danger", "help"
    private String content;

    public DogMessage(String sender, String type, String content) {
        this.sender = sender;
        this.type = type;
        this.content = content;
    }

    public String getSender() { return sender; }
    public String getType() { return type; }
    public String getContent() { return content; }

    @Override
    public String toString() {
        return "[" + sender + "] " + type + ": " + content;
    }
}
```

メッセージキューつきトピック

```
import java.util.concurrent.ConcurrentLinkedQueue;

public class MessageQueueTopic implements Topic {

    private List<Subscriber> subs = new ArrayList<>();
    private ConcurrentLinkedQueue<DogMessage> queue = new ConcurrentLinkedQueue<>();
    private String name;

    public MessageQueueTopic(String name) {
        this.name = name;
    }

    @Override
    public void publish(String message) {
        System.out.println "[" + name + "] publish: " + message);
        for (Subscriber sub : subs) {
            sub.onMessage(name, message);
        }
    }

    public void publishDogMessage(DogMessage msg) {
        queue.add(msg);
        System.out.println "[" + name + "] メッセージ: " + msg);
        for (Subscriber sub : subs) {
            if (sub instanceof DogAgent) {
                ((DogAgent) sub).onDogMessage(msg);
            }
        }
    }

    @Override
    public void subscribe(Subscriber sub) {
        subs.add(sub);
    }

    public boolean hasMessage() {
        return !queue.isEmpty();
    }

    public DogMessage pollMessage() {
        return queue.poll();
    }
}
```

DogAgent（高度な犬エージェント）

```
public interface DogAgent extends Subscriber {  
    void onDogMessage(DogMessage msg);  
    void act();  
    String getName();  
}
```

探索担当：ポチ

```
public class ExplorerDog implements DogAgent {

    private String name;
    private MessageQueueTopic commTopic;
    private int battery = 100;

    public ExplorerDog(String name, MessageQueueTopic commTopic) {
        this.name = name;
        this.commTopic = commTopic;
    }

    @Override
    public String getName() { return name; }

    @Override
    public void act() {
        // 探索する
        System.out.println(name + " : 探索中...");
        battery -= 15;

        // ボールを発見！
        if (Math.random() > 0.7) {
            DogMessage found = new DogMessage(name, "found_ball",
                "ボールを発見！位置は (12, 34)");
            commTopic.publishDogMessage(found);
        }

        // バッテリー残量が少ない
        if (battery < 20) {
            DogMessage help = new DogMessage(name, "help",
                "バッテリー切れ寸前！場所は (15, 20)");
            commTopic.publishDogMessage(help);
        }
    }

    @Override
    public void onDogMessage(DogMessage msg) {
        if (msg.getType().equals("danger")) {
            System.out.println(name + " : 危険！逃げろ！！");
        }
    }

    @Override
    public void onMessage(String topic, String message) {
        // 通常のテキストメッセージ
    }
}
```

運搬担当：Aibo-X

```
public class CarrierDog implements DogAgent {

    private String name;
    private MessageQueueTopic commTopic;
    private boolean carrying = false;
    private int battery = 100;

    public CarrierDog(String name, MessageQueueTopic commTopic) {
        this.name = name;
        this.commTopic = commTopic;
    }

    @Override
    public String getName() { return name; }

    @Override
    public void act() {
        if (carrying) {
            System.out.println(name + " : 荷物を運搬中...");
            battery -= 10;
        } else {
            System.out.println(name + " : 待機中...");
            battery += 5;
        }
    }

    @Override
    public void onDogMessage(DogMessage msg) {
        if (msg.getType().equals("found_ball")) {
            System.out.println(name + " : ボールを回収に向かう!");
            carrying = true;
            DogMessage reply = new DogMessage(name, "going_to_pickup",
                "ボールを回収に行きます");
            commTopic.publishDogMessage(reply);
        }
    }

    @Override
    public void onMessage(String topic, String message) {}
}
```

警備担当：タロウ

```
public class GuardDog implements DogAgent {

    private String name;
    private MessageQueueTopic commTopic;

    public GuardDog(String name, MessageQueueTopic commTopic) {
        this.name = name;
        this.commTopic = commTopic;
    }

    @Override
    public String getName() { return name; }

    @Override
    public void act() {
        // 警戒中
        System.out.println(name + "：周囲を警戒中...");

        // 異常を検知
        if (Math.random() > 0.8) {
            DogMessage danger = new DogMessage(name, "danger",
                "不審な動きを検知!");
            commTopic.publishDogMessage(danger);
        }
    }

    @Override
    public void onDogMessage(DogMessage msg) {
        if (msg.getType().equals("help")) {
            System.out.println(name + "：" + msg.getSender() +
                " が助けを求めている！向かう！");
        }
    }

    @Override
    public void onMessage(String topic, String message) {}
}
```

メイン：3 匹の協調動作

```
public class Main {
    public static void main(String[] args) {

        MessageQueueTopic commTopic = new MessageQueueTopic("communication");

        DogAgent pochi = new ExplorerDog("ポチ", commTopic);
        DogAgent aiboX = new CarrierDog("Aibo-X", commTopic);
        DogAgent taro = new GuardDog("タロウ", commTopic);

        commTopic.subscribe(pochi);
        commTopic.subscribe(aiboX);
        commTopic.subscribe(taro);

        // 10 ステップのシミュレーション
        for (int step = 1; step <= 10; step++) {
            System.out.println("\n=== ステップ " + step + " ===");

            pochi.act();
            aiboX.act();
            taro.act();

            try { Thread.sleep(500); } catch (Exception e) {}
        }
    }
}
```

3. マルチエージェントシステムの設計原則

1. 各エージェントは独立している

- ポチが止まっても Aibo-X は動き続けられる

2. エージェントはメッセージを通してのみ通信する

- 直接メソッドを呼ばない。トピック経由で情報を共有

3. エージェントは自分の役割だけに集中する

- ポチは探索だけ、Aibo-X は運搬だけ、タロウは警備だけ

4. メッセージは非同期で届く

- 送信と受信のタイミングは独立している

まとめ

- マルチエージェントシステムは複数のエージェントが協調する
- 各エージェントは自分の役割を持ち、独立して動く
- メッセージ（トピック通信）で情報を共有する
- イベント駆動で他のエージェントの行動に反応できる
- これは ROS を使った実世界のロボットシステムと同じ設計

キーワード

マルチエージェント 協調動作 メッセージ通信 イベント駆動 役割分担 ROS

演習問題

1. 新しい役割を持つ犬エージェント「医療担当（MedicDog）」を作成してください（バッテリーが切れかけのエージェントに充電スポットの場所を知らせる）。
2. 各エージェントの行動ログを収集する `LoggingAgent` を作成し、すべての通信を監視できるようにしてください。
3. 3匹のエージェントが協力して「ボールを探す → 回収する → 拠点に戻る」という一連のタスクを完了するシミュレーションを完成させてください。

用語集

用語	英語	意味	登場 課
クラス	class	オブジェクトの設計図。プロパティとメソッドを定義する	1
オブジェクト	object	クラスから作られた実際のデータのかたまり（インスタンス）	1
インスタンス	instance	クラスから生成されたオブジェクトのこと	1
インスタンス化	instantiation	<code>new</code> を使ってオブジェクトを作ること	1
プロパティ	property	オブジェクトが持つデータ（属性）	1
メソッド	method	オブジェクトが持つ処理（関数）	1
カプセル化	encapsulation	データを隠して必要な操作だけ公開する設計思想	2
アクセス修飾子	access modifier	アクセス範囲を制御するキーワード（private, public など）	2
private	private	同じクラスの中からしかアクセスできない	2
public	public	どこからでもアクセスできる	2
protected	protected	同じパッケージと子クラスからアクセスできる	4
getter	getter	プロパティの値を取得するメソッド	2
setter	setter	プロパティの値を設定するメソッド	2
バリデーション	validation	値が正しい範囲かどうかをチェックすること	2
コンストラクタ	constructor	オブジェクト生成時に自動実行されるメソッド	3
this	this	自分自身のオブジェクトを指すキーワード	3
オーバーロード	overload	同じ名前で引数の異なるメソッドを複数定義すること	3
継承	inheritance	既存のクラスの機能を引き継いで新しいクラスを作ること	4
extends	extends	継承を宣言するキーワード	4
スーパークラス	superclass	親クラス（継承の元になるクラス）	4
サブクラス	subclass	子クラス（継承して作られたクラス）	4
オーバーライド	override	親クラスのメソッドを子クラスで上書きすること	4
super	super	親クラスを指すキーワード	4
インターフェース	interface	メソッドのシグネチャだけを定義した抽象的な型	5

用語	英語	意味	登場 課
implements	implements	インターフェースを実装することを宣言するキーワード	5
抽象メソッド	abstract method	中身（実装）がないメソッド	5
多態性	polymorphism	同じ操作でもオブジェクトによって振る舞いが変わる仕組み	6
アップキャスト	upcast	子クラスの型を親クラスの型に変換すること	6
動的ディスパッチ	dynamic dispatch	実行時にどのメソッドが呼ばれるかが決まる仕組み	6
戦略パターン	strategy pattern	アルゴリズムをクラス化して差し替え可能にするデザインパターン	7
依存性注入	dependency injection	外部から依存オブジェクトを受け取る設計手法	7
コンポジション	composition	オブジェクトを部品として組み合わせること	7
AI エージェント	AI agent	環境を認識し、意思決定して行動するプログラム	8
状態	state	ある時点におけるシステムの状況	8
行動	action	エージェントが実行する動作	8
世界モデル	world model	行動が状態をどう変えるかを予測するモデル	9
ROS	Robot Operating System	ロボットソフトウェア開発のためのフレームワーク	10
ノード	node	ROS における独立した機能モジュール	10
トピック	topic	ノード間でメッセージをやりとりする通信路	10
publish	publish	トピックにメッセージを送信すること	10
subscribe	subscribe	トピックからメッセージを受信すること	10
疎結合	loose coupling	モジュール間の依存関係が弱い設計	10
マルチエージェント	multi-agent	複数のエージェントが協調して動作するシステム	11
イベント駆動	event-driven	イベントの発生に応じて処理を実行する方式	11

演習問題解答

第0課：Java によるこそ

1. 自分の名前を表示するプログラム

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("私の名前はロボット研究所の田中です");  
    }  
}
```

2. ロボット犬の状態を変数で宣言

```
public class Main {  
    public static void main(String[] args) {  
        String name = "Aibo-X";  
        int battery = 85;  
        double hours = 3.5;  
        System.out.println("名前: " + name);  
        System.out.println("バッテリー: " + battery + "%");  
        System.out.println("稼働時間: " + hours + "時間");  
    }  
}
```

3. println と print の違い

- `println`: 表示したあと改行する
- `print`: 表示したあと改行しない

```
System.out.print("改行"); System.out.print("なし"); // 改行なし  
System.out.println("改行"); System.out.println("あり"); // 改行あり
```

第1課：クラスとオブジェクト

1. charge() メソッドを追加

```
void charge() {  
    battery = 100;  
    System.out.println(name + "：充電完了！");  
}
```

2. bark() でバッテリー消費

```
void bark() {  
    if (battery >= 5) {  
        System.out.println(name + "：ワンワン！");  
        battery -= 5;  
    } else {  
        System.out.println(name + "：バッテリー切れで吠えられません");  
    }  
}
```

3. 3体のロボット犬を作る

```
RobotDog d1 = new RobotDog();  
d1.name = "Aibo-X";  
RobotDog d2 = new RobotDog();  
d2.name = "ポチ";  
RobotDog d3 = new RobotDog();  
d3.name = "タロウ";  
d1.bark(); d2.bark(); d3.bark();
```

第2課：カプセル化

1. バッテリーの範囲チェック setter

```
public void setBattery(int battery) {  
    if (battery < 0) {  
        this.battery = 0;  
    } else if (battery > 100) {  
        this.battery = 100;  
    } else {  
        this.battery = battery;  
    }  
}
```

2. run() メソッド

```
public void run() {
    if (battery >= 25) {
        System.out.println(name + "：走っている！");
        battery -= 25;
    } else {
        System.out.println(name + "：バッテリー不足で走れません");
    }
}
```

3. バッテリー残量警告

```
public void walk() {
    if (battery >= 10) {
        System.out.println(name + "：歩いている...");
        battery -= 10;
        if (battery < 20) {
            System.out.println("⚠ バッテリー残量が" + battery + "%です。充電してください");
        }
    } else {
        System.out.println(name + "：バッテリー切れ！");
    }
}
```

第3課：コンストラクタと初期化

1. color プロパティとコンストラクタ

```
private String color;

public RobotDog(String name, int battery, String color) {
    this.name = name;
    setBattery(battery);
    this.color = color;
}
```

2. デフォルトコンストラクタ

```
public RobotDog() {
    this("名無しロボット", 50, "白");
}
```

3. コンストラクタでのバリデーション

```
public RobotDog(String name, int battery, String color) {
    this.name = name;
    if (battery < 0 || battery > 100) {
        throw new IllegalArgumentException("バッテリーは0~100の範囲で指定してください");
    }
    this.battery = battery;
    this.color = color;
}
```

第4課：継承

1. FlyingRobotDog

```
public class FlyingRobotDog extends SuperRobotDog {
    public FlyingRobotDog(String name) {
        super(name);
    }
    public void hover() {
        if (getBattery() >= 20) {
            System.out.println(getName() + "：ホバリング中...");
            setBattery(getBattery() - 20);
        }
    }
}
```

2. 飛行中の bark() オーバーライド

```
@Override
public void bark() {
    if (flyMode) {
        System.out.println(getName() + "：フライト中は吠えられません");
    } else {
        super.bark();
    }
}
```

3. 飛行中の消費バッテリー半減

```
@Override
public void walk() {
    if (flyMode) {
        if (getBattery() >= 5) {
            System.out.println(getName() + "：飛行しながら移動中...");
            setBattery(getBattery() - 5);
        }
    } else {
        super.walk();
    }
}
```

第5課：インターフェース

1. Swimmable インターフェース

```
public interface Swimmable {
    void swim();
}

public class RobotDog implements Dog, Swimmable {
    public void swim() {
        System.out.println(name + "：防水モードで泳ぐ");
    }
}
```

2. Sensor インターフェース

```
public interface Sensor {
    double readDistance();
}

public class RobotDog implements Dog, Sensor {
    public double readDistance() {
        return Math.random() * 500; // 0~500cm
    }
}
```

3. 「お手」を共通インターフェースで

インターフェースに `hand()` を追加し、両方のクラスで実装する。

```
void hand(); // インターフェースに追加

// RealDog
public void hand() {
    System.out.println(name + " : お手! (本物の犬)");
}
```

第6課：多態性

1. eat() の追加

```
public interface Dog {
    void eat();
}

// RealDog
public void eat() {
    System.out.println(name + " : ご飯を食べる");
}

// RobotDog
public void eat() {
    System.out.println(name + " : 充電ドックに接続");
}
```

2. if-else 地獄とその問題

```
void makeSound(Object animal) {
    if (animal instanceof RealDog) ((RealDog)animal).bark();
    else if (animal instanceof RobotDog) ((RobotDog)animal).bark();
    // 新しい種類が増えるたびに修正が必要
}
// 問題: 拡張性が低い、コードが読みにくい、バグが入りやすい
```

3. List で多態性

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new RealDog("ポチ"));
dogs.add(new RobotDog("Aibo-X"));
dogs.add(new SuperRobotDog("タロウ"));
for (Dog d : dogs) {
    d.bark();
}
```

第7課：戦略パターン

1. SleepyBrain

```
public class SleepyBrain implements Brain {  
    public String decide(String situation) {  
        return "眠いから寝る";  
    }  
}
```

2. AdaptiveBrain

```
public class AdaptiveBrain implements Brain {  
    private RobotDogAgent agent;  
    public AdaptiveBrain(RobotDogAgent agent) { this.agent = agent; }  
    public String decide(String situation) {  
        if (agent.getBattery() < 30) {  
            return new CautiousBrain().decide(situation);  
        }  
        return new CuriousBrain().decide(situation);  
    }  
}
```

第8課：頭脳差し替え

1. AggressiveBrain

```
public class AggressiveBrain implements Brain {  
    public Action decide(State state) {  
        if (state.getBattery() >= 50) return new SearchAction();  
        return new ChargeAction();  
    }  
}
```

2. temperature の追加と頭脳

State に `int temperature` を追加。温度が 35 度以上のときは充電を控える。

3. LoggingBrain

```

public class LoggingBrain implements Brain {
    private Brain wrapped;
    private List<String> history = new ArrayList<>();
    public LoggingBrain(Brain wrapped) { this.wrapped = wrapped; }
    public Action decide(State state) {
        Action a = wrapped.decide(state);
        history.add(state + " → " + a.getName());
        return a;
    }
}

```

第9課：世界モデル

1. 安定性の評価軸を追加

```

public double evaluate(State predicted) {
    double score = predicted.getBattery() * 0.5;
    score -= Math.abs(predicted.getBattery() - 50) * 0.3; // 50% 付近を安定と評価
    return score;
}

```

2. TwoStepPredictiveBrain

```

public Action decide(State current) {
    Action bestAction = null;
    double bestScore = Double.NEGATIVE_INFINITY;
    for (Action first : actions) {
        State afterFirst = model.predict(current, first);
        for (Action second : actions) {
            State afterSecond = model.predict(afterFirst, second);
            double score = model.evaluate(afterSecond);
            if (score > bestScore) { bestScore = score; bestAction = first; }
        }
    }
    return bestAction;
}

```

第10課：pub/sub

1. play トピックの追加

```

Topic playTopic = new BroadcastTopic("play");
// DogNode に play メソッド追加
public void play() {
    playTopic.publish(name + "：遊んでいる！");
}

```

2. LoggerNode

```

public class LoggerNode implements Subscriber {
    private List<String> log = new ArrayList<>();
    public void onMessage(String topic, String msg) {
        String entry = "[" + java.time.LocalDateTime.now() + "] " + topic + ": " + msg;
        log.add(entry);
        System.out.println(" [ログ] " + entry);
    }
}

```

第11課：マルチエージェント

1. MedicDog

```

public class MedicDog implements DogAgent {
    public void onDogMessage(DogMessage msg) {
        if (msg.getType().equals("help")) {
            System.out.println(name + "：充電スポットの場所を案内します！");
            commTopic.publishDogMessage(new DogMessage(name, "info",
                "充電スポットは座標 (0, 0) にあります"));
        }
    }
}

```

2. LoggingAgent

```

public class LoggingAgent implements Subscriber {
    private List<String> log = new ArrayList<>();
    public void onMessage(String topic, String msg) {
        log.add(topic + ": " + msg);
    }
    public void printLog() {
        log.forEach(System.out::println);
    }
}

```

コード索引

この索引は、教科書に登場するすべてのコード例をクラス名・ファイル名から検索できます。

主要クラス一覧

クラス/インターフェース	課	説明
Dog (interface)	5, 6	犬の共通インターフェース
RobotDog	1, 2, 3, 5	ロボット犬クラス
RealDog	5, 6	本物の犬クラス
SuperRobotDog	4	スーパーロボット犬（継承）
FlyingRobotDog	4 (演習)	飛行ロボット犬
Brain (interface)	7, 8	頭脳インターフェース
CuriousBrain	7	好奇心旺盛な頭脳
CautiousBrain	7	慎重な頭脳
RandomBrain	7	ランダムな頭脳
SimpleBrain	8	シンプルなルールベース頭脳
PredictiveBrain	9	世界モデル搭載の予測頭脳
State	8, 9	ロボットの状態を表す
Action	8, 9	ロボットの行動を表す
WorldModel (interface)	9	世界モデルインターフェース
SimpleWorldModel	9	シンプルな世界モデル
RobotDogAgent	7, 8	頭脳を持ったエージェント
Subscriber (interface)	10	購読者インターフェース
Topic (interface)	10	トピックインターフェース
BroadcastTopic	10	ブロードキャストトピック
MessageQueueTopic	11	メッセージキューつきトピック
DogNode	10	犬ノード（pub/sub）
DisplayNode	10	表示ノード
DogAgent (interface)	11	高度な犬エージェント
ExplorerDog	11	探索担当エージェント
CarrierDog	11	運搬担当エージェント

クラス/インターフェース	課	説明
GuardDog	11	警備担当エージェント
DogMessage	11	エージェント間メッセージ

設計パターン一覧

パターン	課	説明
getter / setter	2	カプセル化の基本パターン
コンストラクタオーバーロード	3	複数の初期化パターン
継承 (extends)	4	既存クラスの機能拡張
メソッドオーバーライド	4	親のメソッドを上書き
インターフェース実装 (implements)	5	能力の約束
アップキャスト	6	具象型→抽象型への変換
動的ディスパッチ	6	実行時のメソッド解決
戦略パターン (Strategy)	7	頭脳の差し替え
依存性注入 (DI)	7	外部からの部品注入
コンポジション	7	部品の組み合わせ
pub/sub パターン	10	メッセージ通信
マルチエージェント	11	複数エージェントの協調

コードカテゴリ（学習目的別）

Java 文法を学びたい

- 変数と型 → 第0課
- クラス定義 → 第1課
- メソッド → 第1課
- コンストラクタ → 第3課
- アクセス修飾子 → 第2課

OOP 三大要素を学びたい

- カプセル化 → 第2課
- 継承 → 第4課
- 多態性 → 第6課

設計力を高めたい

- インターフェース設計 → 第5課
- 戦略パターン → 第7課
- pub/sub アーキテクチャ → 第10課
- マルチエージェント設計 → 第11課

ロボティクスを学びたい

- AI エージェント → 第8課
- 世界モデル → 第9課
- ROS 風通信 → 第10課
- マルチエージェント協調 → 第11課

教師用ガイド

教科書の設計思想

本書は「ロボット犬」という身近で具体的な題材を通じて、Java とオブジェクト指向プログラミング（OOP）を学ぶ教科書です。

3つの学習レイヤー

レイヤー3：ロボティクス設計（世界モデル・ROS・マルチエージェント）
レイヤー2：OOP 設計（継承・インターフェース・多態性・戦略パターン）
レイヤー1：Java 文法（クラス・変数・メソッド・コンストラクタ）

各課はこの3層を同時に学べるように設計されています。

授業進行の提案

標準カリキュラム（全15回）

回	課	授業の進め方	準備物
1	0	Java 環境の説明、プレイグラウンドの使い方	ブラウザ
2	1	クラスとオブジェクトの概念、実際にコードを書かせる	IDE or プレイグラウンド
3	2	「バッテリーがマイナスになる」問題を提示 → カプセル化を導入	前回のコード
4	3	コンストラクタの便利さを実感させる演習	—
5	4	「スーパーロボット犬」のストーリーで継続を導入	—
6	5	リアル犬とロボット犬の違いからインターフェースへ	—
7	6	多態性のデモ（if-else 地獄を見せてから）	スライドがあると効果的
8	7	戦略パターン、頭脳差し替えのライブコーディング	—
9	8	AI エージェント完全実装、グループ演習	—
10	9	世界モデルの概念、予測のコード	—
11	10	pub/sub、ROS の概念説明	図解があると良い
12	11	マルチエージェント、3匹の協調デモ	—
13	—	総合演習（自由課題の着手）	—
14	—	自由課題発表準備	—
15	—	発表会・まとめ	—

短縮カリキュラム（全8回）

回	内容
1	第0課 + 第1課（Java 導入 + クラスとオブジェクト）
2	第2課 + 第3課（カプセル化 + コンストラクタ）
3	第4課（継承）
4	第5課 + 第6課（インターフェース + 多態性）
5	第7課（戦略パターン）
6	第8課（AI エージェント）
7	第9課（世界モデル）
8	第10課 + 第11課（ROS + マルチエージェント）

各課の指導ポイント

第0課：Java によるこそ

- **最重要ポイント:** プレイグラウンドで「動いた！」という体験をさせる
- **注意点:** 環境構築でつまずくとやる気が下がる。最初はブラウザ上のプレイグラウンドを使うのが安全
- **発展課題:** System.out.println で ASCII アートのロボット犬を描く

第1課：クラスとオブジェクト

- **最重要ポイント:** 「設計図」と「実物」の違いを物理的な例えで理解させる
- **注意点:** `new` キーワードの意味が抽象的なので、繰り返し説明する
- **アクティビティ:** 教室で「人間クラス」から「生徒オブジェクト」を作るロールプレイ

第2課：カプセル化

- **最重要ポイント:** 「なぜ隠す必要があるのか」を実感させる
- **注意点:** getter/setter の「おまじない感」を減らすために、ないと何が困るかを先に体験させる
- **アクティビティ:** 悪意のあるコードでバッテリーを -999 にする「攻撃」をやってみせる

第3課：コンストラクタ

- **最重要ポイント:** 「忘れずに初期化する」ための仕組み
- **発展課題:** コンストラクタ内でファイル読み込みやネットワーク接続を行う例

第4課：継承

- **最重要ポイント:** 「is-a」関係の理解
- **アクティビティ:** クラスの階層構造をホワイトボードに書かせる
- **落とし穴:** 継承よりコンポジションの話はここでは深入りしない（第7課で扱う）

第5課：インターフェース

- **最重要ポイント:** 「約束事」としてのインターフェース
- **注意点:** 継承との違いが曖昧になりがちなので、表で比較する
- **アクティビティ:** 犬以外の動物（Cat, Bird）でも Dog インターフェースを実装させてみる

第6課：多態性

- **最重要ポイント:** この課が OOP のクライマックス
- **アクティビティ:** if-else で書いたコードを多態性で書き換えるリファクタリング演習
- **発展課題:** `instanceof` を使うべきでない理由を議論する

第7課：戦略パターン

- **最重要ポイント:** 「継承よりコンポジション」の実践
- **アクティビティ:** グループごとに異なる頭脳を実装して、同じエージェントで動かす
- **発展課題:** デコレーターパターン（LoggingBrain）への発展

第8課：AI エージェント

- **最重要ポイント:** 状態・行動・頭脳の3要素
- **アクティビティ:** ロボット掃除機（Roomba）を例にしたエージェント設計
- **発展課題:** 強化学習とのつながりを紹介

第9課：世界モデル

- **最重要ポイント:** 「予測」という思考の導入

- **注意点:** ここで挫折する生徒が出ないように、シンプルな評価関数から始める
- **発展課題:** 複数ステップの先読み、MinMax アルゴリズムへの発展

第10課：pub/sub

- **最重要ポイント:** 疎結合の価値
- **アクティビティ:** 実際の ROS の画面を見せる（rviz など）
- **発展課題:** MQTT などの他のメッセージングシステムとの比較

第11課：マルチエージェント

- **最重要ポイント:** 「協調」と「役割分担」
- **アクティビティ:** クラス全体で一つのシミュレーションを動かす
- **発展課題:** 競合（リソースの取り合い）の実装

自由課題のアイデア

1. **ロボット犬シミュレータ** 複数の頭脳・複数の犬・複数のトピックを組み合わせた完全なシミュレータ
2. **迷路探索ロボット** 壁センサーと世界モデルを使って迷路を探索するロボット
3. **犬のしつけシミュレータ** 報酬と罰を使って犬の行動を学習させる（Q-learning の導入）
4. **災害救助チーム** 複数のロボット犬が協力して被災者を探すシミュレーション

評価のポイント

評価項目	割合	評価基準
演習問題	40%	各課末の問題。コードが正しく動作すること
総合演習	30%	自由課題。完成度 + 工夫 + コードの品質
発表	30%	プレゼンテーション + 質疑応答

コード品質の評価基準

- カプセル化が適切に使われているか
- 継承・インターフェースが適切に使われているか

- 多態性を活用できているか
- 無駄な重複コードがないか
- わかりやすい名前が使われているか

トラブルシューティング

よくあるつまずき

症状	原因	対処
<code>static</code> がわからない	静的メソッドとインスタンスメソッドの混同	「クラスに属する vs オブジェクトに属する」の図解
<code>this</code> の意味がわからない	自分自身の参照概念が難しい	「クラスの中から自分を見る」という比喻
継承とインターフェースの違いがわからない	抽象概念のオーバーロード	表で比較、具体例を増やす
コンパイルエラーが直せない	エラーメッセージが読めない	代表的なエラーと対処法の一覧表を配布